



SIMATS ENGINEERING
CO5-ASSESSMENT TOOL -2
Saveetha Institute of Medical and Technical Sciences
Chennai- 602105



Student Name: R.Gautham Krishna

Reg. No.: 192421128

Course Code: CSA0613

Slot: A

Course Name: DESIGN AND ANALYSIS OF ALGORITHMS

Course Faculty: Dr. Mary Harin Fernandez F

Q28. Edge Coloring Problem Using Backtracking

1. Title

Edge Coloring Problem Using Backtracking

2. Problem Statement

Given a graph, color its edges using a limited number of colors such that no two adjacent edges have the same color.

The algorithm uses Backtracking to try different colors and pruning to reject invalid assignments.

3. Objective

The objectives are:

1. To color all edges of a graph.
2. To ensure adjacent edges have different colors.
3. To use Backtracking.
4. To apply pruning for invalid assignments.
5. To generate a valid edge coloring.
6. To analyze the algorithm complexity.

4. Input

The algorithm accepts:

- Graph $G = (V, E)$
- Number of vertices: V
- Number of edges: E
- Number of colors: K

Example

Vertices = $\{A, B, C, D\}$

Edges = $\{$

$(A,B), (A,C), (B,C), (B,D)$

$\}$

Colors = $\{1, 2, 3\}$

5. Output

A valid color assignment for every edge.

Example Output

$(A,B) \rightarrow \text{Color 1}$

$(A,C) \rightarrow \text{Color 2}$

$(B,C) \rightarrow \text{Color 3}$

$(B,D) \rightarrow \text{Color 2}$

No two edges sharing the same vertex have the same color.

6. Concept of Backtracking

Backtracking works by:

1. Selecting an edge.
2. Trying a color.
3. Checking whether the color is valid.
4. Recursively coloring the next edge.
5. Removing the color if it causes a conflict.
6. Trying another color.

7. Main Pseudocode

Edge Coloring Using Backtracking

ALGORITHM EdgeColoring(G, K)

INPUT:

G = Graph

K = Number of colors

OUTPUT:

Color assigned to every edge

BEGIN

color[] $\leftarrow$ Empty

CALL Backtrack(0)

IF solution found

```
    PRINT color[]  
ELSE  
    PRINT "No valid coloring"  
END
```

8. Backtracking Pseudocode

```
PROCEDURE Backtrack(index)  
  
    IF index = Number of edges THEN  
        RETURN TRUE  
    END IF  
  
    FOR  $c \leftarrow 1$  TO  $K$  DO  
  
        IF IsSafe(index,  $c$ ) THEN  
  
            color[index]  $\leftarrow c$   
  
            IF Backtrack(index + 1) THEN  
                RETURN TRUE  
            END IF  
  
            color[index]  $\leftarrow 0$ 
```

END IF

END FOR

RETURN FALSE

END PROCEDURE

9. Safe Color Check

FUNCTION IsSafe(edge, c)

FOR every previous edge e DO

IF e and edge share a vertex

AND $\text{color}[e] = c$ THEN

RETURN FALSE

END IF

END FOR

RETURN TRUE

END FUNCTION

If two edges are connected to the same vertex, they are adjacent edges and must have different colors.

10. Complete Pseudocode

ALGORITHM EdgeColoring(G, K)

INPUT:

$G = (V, E)$

K = Number of colors

OUTPUT:

Color of every edge

BEGIN

color[0... $E-1$] $\leftarrow 0$

IF Backtrack(0) = TRUE THEN

 PRINT color

ELSE

 PRINT "No valid coloring"

END

PROCEDURE Backtrack(index)

IF index = E THEN

 RETURN TRUE

END IF

FOR $c \leftarrow 1$ TO K DO

 IF IsSafe(index, c) THEN

 color[index] $\leftarrow$ c

 IF Backtrack(index + 1) THEN

 RETURN TRUE

 END IF

 color[index] $\leftarrow$ 0

 END IF

END FOR

RETURN FALSE

END PROCEDURE

11. Step-by-Step Working

Consider:

Edges:

(A,B)

(A,C)

(B,C)

(B,D)

Available colors:

1, 2, 3

Step 1

Assign Color 1 to:

(A,B) $\rightarrow$ 1

Step 2

(A,C) shares vertex A with (A,B), so it cannot use Color 1.

(A,C) $\rightarrow$ 2

Step 3

(B,C) is adjacent to both previous edges.

(B,C) $\rightarrow$ 3

Step 4

(B,D) shares B with (A,B) and (B,C).

Color 2 is available:

(B,D) $\rightarrow$ 2

Final coloring:

$(A,B) \rightarrow 1$

$(A,C) \rightarrow 2$

$(B,C) \rightarrow 3$

$(B,D) \rightarrow 2$

12. Pruning

Pruning avoids unnecessary choices.

If a selected color is already used by an adjacent edge:

IF adjacent edge has same color

REJECT color

The algorithm immediately moves to the next color.

If no color is available, it backtracks to the previous edge and changes its color.

13. Handling All Cases

Case 1: Empty Graph

If there are no edges:

Output: No edges to color

Case 2: One Edge

One color is sufficient.

$(A,B) \rightarrow \text{Color 1}$

Case 3: Sufficient Colors

A valid coloring can be found.

Case 4: Insufficient Colors

If the available colors are insufficient:

Output: No valid coloring

14. Correctness

The algorithm is correct because:

- Every edge is considered.
- Only valid colors are assigned.
- Adjacent edges never receive the same color.
- Invalid choices are rejected.
- Backtracking explores alternative colors.

Therefore, if a valid coloring exists with K colors, the algorithm can find it.

15. Algorithm Analysis

Let:

- E = Number of edges
- K = Number of colors

For each edge, the algorithm may try K colors.

In the worst case:

Time Complexity – $O(K^E \times E)$

Space Complexity – $O(E)$

The pruning condition reduces unnecessary searches in practice.

16. Applications

Edge coloring is useful in:

- Scheduling: Avoiding conflicts between tasks.

- Communication networks: Assigning different frequencies to connected links.
- Timetabling: Avoiding overlapping activities.
- Channel assignment: Assigning communication channels.
- Transportation: Managing conflicting routes.

17. Advantages

- Simple Backtracking approach.
- Avoids conflicts between adjacent edges.
- Finds a valid coloring if one exists.
- Pruning reduces unnecessary computation.
- Can handle different numbers of colors.

18. Limitations

- Can be slow for large graphs.
- Number of possible color assignments increases rapidly.
- Worst-case complexity is exponential.
- Not ideal for very large graphs.

19. Result

For the graph:

(A,B), (A,C), (B,C), (B,D)

using 3 colors, a valid coloring is:

(A,B) $\rightarrow$ 1

(A,C) $\rightarrow$ 2

$(B,C) \rightarrow 3$

$(B,D) \rightarrow 2$

All adjacent edges have different colors.

20. Conclusion

The Edge Coloring Problem was successfully solved using Backtracking. The algorithm assigns colors to edges while ensuring that adjacent edges have different colors. Pruning rejects invalid color assignments and reduces unnecessary searches.

The approach is useful for scheduling, communication networks, channel allocation, and other conflict-management problems.

Time Complexity – $O(K^E \times E)$ | Space Complexity – $O(E)$